

P3798R1

The unexpected in `std::expected`

Published Proposal, 2025-08-01

This version:

<https://godexsoft.github.io/papers/P3798R1.html>

Authors:

[Alex Kremer](#) (Ripple)

[Ayaz Salikhov](#) (Ripple)

Audience:

LEWG, LWG

Project:

ISO/IEC 14882 Programming Languages — C++, ISO/IEC JTC1/SC22/WG21

Source:

<https://github.com/godexsoft/papers/blob/master/source/P3798R1.bs>

Abstract

We propose adding a `has_error()` member function to `std::expected` to complement the existing `has_value()` functionality.

Table of Contents

1 Revision History

1.1 R1

2 Motivation

2.1 Sample usecase

2.2 Impact on the standard

2.3 Other languages

3 Proposed Wording

3.1 Feature test macro

3.2 Expected object

3.3 Expected void

4 Acknowledgements

References

Informative References

§ 1. Revision History

§ 1.1. R1

- Example now uses `std::expected<void, std::string>`
- Use shorter wording as suggested by Daniel Krügler
- Added SG10 recommendation to bump `__cpp_lib_expected`
- Added acknowledgements

§ 2. Motivation

Today, `std::expected` provides a `has_value()` member function that can be used to check whether the instance holds a value or an error. There is also an implicit conversion operator to `bool` that can be used for the same purpose. These two existing mechanisms follow several other facilities in the language, including `std::optional`.

While `std::optional` provides only the `has_value()` member function as its primary state-checking mechanism, `std::expected` serves a fundamentally different purpose. Unlike `std::optional`, which represents either a value or nothing, `std::expected` represents either a value or an error. This semantic difference warrants distinct interface considerations. Adding `has_error()` creates symmetry in the API that better reflects the dual-state nature of `std::expected` and provides more readable, self-documenting code when the focus is on error handling rather than value presence.

In case of `void` partial specialization of `std::expected`, `has_value()` member function is obscure, because `void` doesn't represent a value on its own. In the meantime `has_error()` would be clear and more understandable.

§ 2.1. Sample usecase

Consider the following examples (assuming `result` is of type `std::expected<void, std::string>`):

Without this proposal	With this proposal
<pre>if (!result.has_value()) log_and_exit(result.error());</pre>	<pre>if (result.has_error()) log_and_exit(result.error());</pre>
<pre>// somewhere in unit tests ASSERT_TRUE(!result.has_value()); EXPECT_EQ(result.error(), "myError");</pre>	<pre>// somewhere in unit tests ASSERT_TRUE(result.has_error()); EXPECT_EQ(result.error(), "myError");</pre>

§ 2.2. Impact on the standard

This change is entirely based on library extensions and does not require any language features beyond what is available in C++ 23.

§ 2.3. Other languages

It might be unexpected to not have a `has_error()` method in `std::expected`, especially when moving from other languages that have similar constructs.

The proposed approach is consistent with similar facilities in some other programming languages:

- **Rust** provides both `is_ok()` and `is_err()` methods on its `Result` type.
- **D** provides both `hasValue` and `hasError` properties in its `expected` package.
- **Haskell** provides `isRight` and `isLeft` in its `Either` type.

These implementations acknowledge the dual-state nature of error-handling types by offering explicit methods for checking both states, rather than relying solely on negation of a value-checking method.

§ 3. Proposed Wording

§ 3.1. Feature test macro

Bump the `__cpp_lib_expected` value in 17.3.2 [version.syn]:

```
#define __cpp_lib_execution      201902L // also in <execution>
#define __cpp_lib_expected      202211L // also in <expected>
#define __cpp_lib_expected      20yymmL // also in <expected>
#define __cpp_lib_filesystem    201703L // also in <filesystem>
```

§ 3.2. Expected object

In 22.8.6.1 [expected.object.general] add:

```
constexpr bool has_value() const noexcept;  
constexpr bool has_error() const noexcept;  
constexpr const T& value() const &;
```

In 22.8.6.6 [expected.object.obs] add after §7:

```
constexpr bool has_error() const noexcept;  
Effects: Equivalent to: return !has_value();
```

§ 3.3. Expected void

In 22.8.7.1 [expected.void.general] add:

```
constexpr bool has_value() const noexcept;  
constexpr bool has_error() const noexcept;  
constexpr void operator*() const noexcept;
```

In 22.8.7.6 [expected.void.obs] add after §1:

```
constexpr bool has_error() const noexcept;  
Effects: Equivalent to: return !has_value();
```

§ 4. Acknowledgements

We would like to thank Bronek Kozicki and Gašper Ažman for reviewing the initial draft and providing early feedback, as well as Daniel Krügler for recommending the use of shorthand syntax for "Equivalent to" in the wording.

§ References

§ Informative References

[P0032R2]

Vicente J. Botet Escriba. *Homogeneous interface for variant, any and optional (Revision 2)*. 13 March 2016. URL: <https://wg21.link/p0032r2>

[P0323R12]

Vicente Botet, JF Bastien, Jonathan Wakely. *std::expected*. 7 January 2022. URL: <https://wg21.link/p0323r12>